VocaCare

Voice-Powered RAG Healthcare Appointment Assistant

End-to-end building plan

A practical, portfolio-ready roadmap for building a voice-first appointment workflow with FastAPI, React, an LLM, RAG, SQL, and a doctor dashboard.

Prototype stack: mostly free and local

1. Product Idea

The core workflow is: Patient speaks naturally -> speech-to-text -> structured LLM extraction -> doctor and hospital retrieval -> exact availability validation -> patient confirmation -> appointment booking -> structured summary for the doctor.

Example extraction

```
{
  "intent": "book_appointment",
  "patient_name": "Rahul",
  "specialization": "Dermatology",
  "preferred_date": "2026-08-19",
  "preferred_time": "17:00",
  "symptoms": ["itching", "red patches on hands"],
  "duration": "1 week",
  "appointment_type": "consultation"
}
```

The doctor should receive the structured appointment summary, not the entire raw voice conversation. The original conversation can remain available behind an explicit View conversation action.

2. Recommended Free or Low-Cost Stack

Layer	Tool	Purpose
Frontend	React + Vite	Patient voice UI and doctor dashboard
Backend	FastAPI + Uvicorn	API endpoints and orchestration
Database	PostgreSQL + SQLAlchemy	Patients, doctors, slots, bookings
Speech-to-text	Whisper local	Convert uploaded audio to text
LLM	Ollama local or a free-tier provider	Intent and structured field extraction
Vector database	ChromaDB local	Doctor, hospital, FAQ retrieval
Embeddings	Sentence Transformers	Local semantic search embeddings
Text-to-speech	Browser SpeechSynthesis	Read responses aloud at no API cost
Validation	Pydantic	Typed, controlled structured output
Development	VS Code + Git + GitHub	Implementation, testing, version control

Start with the smallest useful set: React, FastAPI, PostgreSQL, Whisper, one LLM, ChromaDB, and Sentence Transformers. Add agent frameworks only when the workflow needs them.

3. Architecture

```
PATIENT
   | Voice
   v
React voice recorder
   | POST /voice/transcribe
   v
Whisper speech-to-text
   | transcript
   v
LLM extraction + missing-field logic
   | structured JSON
   +----------------------+
```

```
   |                    |
   v                    v
ChromaDB RAG          PostgreSQL
Doctor/hospital       Exact slots and bookings
profiles and FAQs     Availability source of truth
   +---------------------+
             v
        Appointment tools
              |
              v
 Confirmation + doctor summary
```

Use RAG to answer who or what is relevant. Use PostgreSQL to decide what is actually available and to perform transactional booking. Never let the vector database be the source of truth for appointment slots.

4. VS Code Project Structure

```
voice-rag-appointment/
|-- backend/
|   |-- app/
|       |-- main.py
|       |-- api/ (voice.py, appointments.py, doctors.py, patients.py)
|       |-- ai/ (speech.py, extractor.py, conversation.py, prompts.py)
|       |-- rag/ (embeddings.py, vectorstore.py, retriever.py, ingest.py)
|       |-- database/ (database.py, models.py, schemas.py)
|       |-- services/ (appointment_service.py, doctor_service.py, patient_service.py)
|       |-- utils/ (datetime_utils.py)
|-- frontend/
|   |-- src/
|       |-- components/ (VoiceButton, Conversation, AppointmentCard, DoctorCard)
|       |-- pages/ (PatientHome, DoctorDashboard, Appointment)
|       |-- services/api.js
|-- data/ (doctors.json, hospitals.json, departments.json, faq.json)
|-- chroma_db/
|-- README.md
|-- .env.example
|-- .gitignore
```

5. Build Order

Phase 1: Working foundation

Install Python 3.11 or 3.12, Node.js, Git, PostgreSQL, and VS Code. Create a backend virtual environment, install FastAPI, SQLAlchemy, Pydantic, Uvicorn, and python-multipart. Confirm /docs opens.

Phase 2: Database first

Create patients, doctors, doctor_availability, appointments, medical_context, and conversations. Seed fake doctors and schedules. PostgreSQL remains the booking source of truth.

Phase 3: Normal appointment APIs

Implement GET /doctors, GET /doctors/{id}, GET /doctors/{id}/availability, POST /appointments, GET /appointments/{id}, DELETE /appointments/{id}, and availability checks. Test these before adding AI.

Phase 4: React patient and doctor views

Build doctor browsing, slot selection, booking confirmation, appointment history, and the dashboard. Connect with fetch or Axios and show loading, empty, error, and success states.

Phase 5: Speech

Record audio in the browser, upload it to POST /voice/transcribe, run Whisper locally, and return only the transcript. Test upload and transcription independently.

Phase 6: Structured extraction

Send transcript plus current date and timezone to the LLM. Require JSON matching a Pydantic model. Missing values must be null; the model must not invent details or diagnoses.

Phase 7: Conversation filling

Persist session state. Detect required missing fields, ask one focused question, merge the answer into the schema, and repeat until the appointment request is complete.

Phase 8: RAG

Ingest doctor profiles, hospitals, departments, FAQs, fees, languages, and policies into ChromaDB using local embeddings. Retrieve relevant doctors and information, then check exact slots in SQL.

Phase 9: Tools and confirmation

Expose find_doctors, check_availability, create_appointment, cancel_appointment, and reschedule_appointment as controlled backend functions. Require explicit confirmation before booking.

Phase 10: Full voice loop

Return a concise response to React and use SpeechSynthesis to read it. Finish with an end-to-end demo: speak -> extract -> ask missing fields -> retrieve -> confirm -> book -> doctor summary.

6. Data Design

```
Patient
  id, name, phone, email, age, gender, created_at
Doctor
  id, name, specialization, hospital, experience, consultation_fee
DoctorAvailability
  id, doctor_id, date, start_time, end_time, is_available
Appointment
  id, patient_id, doctor_id, appointment_date, appointment_time, status, symptoms, created_at
MedicalContext
  appointment_id, symptoms, duration, existing_conditions, medications
Conversation
  id, patient_id, appointment_id, role, content, created_at
```

Appointment extraction schema

```
{
  "intent": "book_appointment | find_doctor | cancel | reschedule | other",
  "patient": {"name": null, "age": null, "gender": null, "phone": null},
  "appointment": {
    "specialization": null, "doctor": null, "date": null, "time": null,
    "time_preference": null, "appointment_type": "consultation"
  },
  "medical_context": {
    "symptoms": [], "duration": null,
    "existing_conditions": [], "medications": []
  }
}
```

Use symptoms and patient-stated conditions. Do not create a disease or diagnosis field from an uncertain model guess.

7. API Contract

Method	Endpoint	Responsibility
POST	/voice/transcribe	Audio upload -> transcript
POST	/appointment/extract	Transcript -> validated structured fields
POST	/appointment/search	Specialization/date/time -> matching doctors and SQL slots
POST	/appointment/book	Explicit confirmation -> transactional booking
GET	/doctor/appointments	Doctor dashboard schedule and summaries
GET	/patient/appointments	Patient appointment history
GET	/doctors	Doctor profile search
GET	/doctors/{id}/availability	Exact availability from SQL

8. Missing Information and Agent Flow

For a request such as 'I want to book a doctor's appointment', extraction should return null for name, specialization, date, and time. The conversation service asks only for the next required field, stores the answer, and re-runs extraction against the accumulated context. After retrieval, the assistant offers concrete slots. Booking happens only after the patient says yes or selects a slot.

```
1. Receive text or transcript
2. Extract fields with null for unknown values
```

```
3. Determine the next required field
4. Ask one question and save the answer
5. Repeat until patient, specialty, date, and time are known
6. Retrieve relevant doctors with RAG
7. Check exact availability in PostgreSQL
8. Present choices and request confirmation
9. Call the booking service in a transaction
10. Generate a structured doctor summary
```

9. RAG Responsibilities

RAG is useful for doctor specialization, hospital information, departments, fees, languages, locations, policies, FAQs, and routing hints. It is not appropriate for exact availability, booking, cancellation, or diagnosis. For chest pain or other urgent symptoms, route the patient to appropriate urgent care or emergency guidance instead of claiming a diagnosis.

Ingestion pipeline

```
JSON and text documents
       |
Chunk and normalize
       |
Sentence Transformer embeddings
       |
ChromaDB collection
       |
Top-k retrieval with metadata filters
```

10. Doctor Dashboard

The dashboard should prioritize today's schedule and concise patient context. Each appointment can show time, patient name, symptoms, duration, appointment type, status, and a View conversation action. The raw conversation is secondary; the structured summary is the primary clinical handoff for this prototype.

```
09:00 AM   Rahul Sharma
           Symptoms: itching; red patches on hands
           Duration: 1 week
           Type: consultation
           Status: confirmed

10:30 AM   Priya Patil
           Symptoms: headache; fatigue
           Duration: 3 days
```

11. Safety and Reliability Rules

• Never invent missing patient, date, time, doctor, or symptom information.

• Do not diagnose. Route symptoms to an appropriate department or urgent care.

• Require explicit confirmation before creating an appointment.

• Use PostgreSQL transactions and database constraints to prevent double booking.

• Keep API keys in .env and exclude them from Git.

• Use fake data for the portfolio project; do not ingest real patient data.

• Log request IDs and errors without logging unnecessary sensitive medical content.

• Add authentication and role-based access before any real deployment.

12. Three-Week Roadmap

Week	Days	Deliverable
1	1-2	Project setup, PostgreSQL schema, fake doctors and availability
1	3-4	Appointment APIs and React patient/doctor screens
1	5-7	Frontend integration, booking flow, error states, tests
2	8-9	Whisper upload and browser recording
2	10-12	LLM extraction, Pydantic validation, missing-field engine
2	13-14	Appointment tools and complete voice booking
3	15-18	RAG documents, embeddings, ChromaDB, retrieval
3	19-20	RAG plus SQL doctor search and dashboard summaries
3	21	End-to-end testing, README, demo recording, deployment review

Portfolio positioning

The differentiator is not simply a chatbot. It is the reliable conversion of unstructured voice into a controlled appointment workflow: extraction, missing-field conversation, RAG-assisted routing, transactional availability validation, confirmation, and a useful doctor summary.

Recommended first implementation

> Start with the normal appointment engine and database. Then add voice transcription, structured extraction, conversational field filling, RAG, and finally the full voice response loop. This keeps every milestone demonstrable and makes debugging much easier.